

Nexa

A frontend framework with nothing to install.

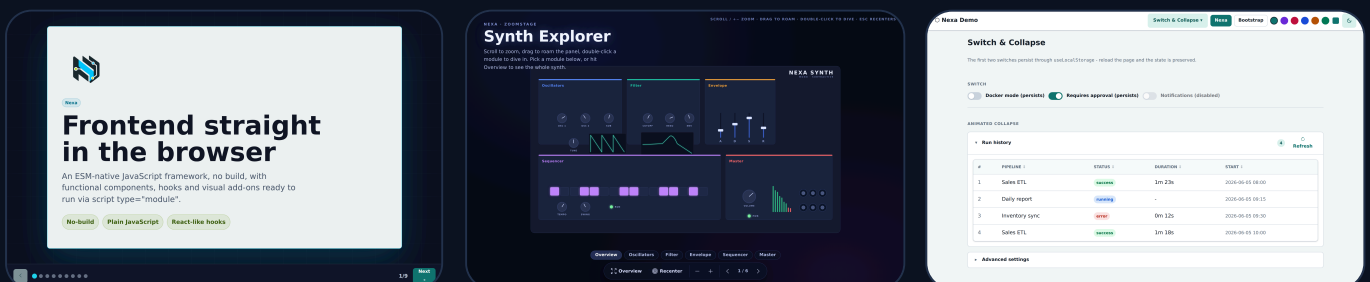
No build. No bundler. No `node_modules`. Nexa is a no-build, ESM-native JavaScript framework with a React-like hooks API — the whole app is plain ES modules the browser runs directly. For a security and architecture review, that single decision is the headline.

Zero runtime dependencies

Runs under strict CSP

~60 components + add-ons

MIT



Above: three interfaces built with Nexa — a zoom presentation, a synthesizer panel, and a component gallery. All served as static files, no build step.

01 Executive summary

What it is. Nexa is a client-side UI framework whose components are plain JavaScript functions and whose runtime is a set of ES modules loaded with `<script type="module">`. It gives a React-trained team a familiar hooks API (`useState`, `useEffect`, `context`, `router`, `SSR`) and ships ~60 UI components, a design system, and visual add-ons — with no compiler in the loop.

Why it's on your desk. Most frontend risk and cost lives in the *toolchain*, not the app: hundreds of transitive npm packages, build plugins that execute at install time, and a bundler you must keep current. Nexa removes that layer entirely. That is simultaneously a **security** story (no supply chain to attack or audit) and an **architecture** story (nothing to rebuild, re-lock, or migrate).

0

runtime
dependencies

0

build steps in
dev

0

eval / dynamic
code

61

UI components

~300

tests × 3
engines

MIT

vendored, you
own it



For Security

- **No supply chain.** Zero third-party runtime code; nothing from npm at runtime or build.
- **CSP-friendly.** No `eval/new Function` — runs under a strict policy without `unsafe-eval`.
- **Auditable.** The whole core is a few readable modules you can vendor and diff.
- **Safe by default.** SSR output is HTML-escaped; a `safeUrl()` guard ships in-box.



For Architecture

- **No build.** Dev is F5 on plain ES modules; deploy is copying static files.
- **Longevity.** No bundler/transpiler to age out; the platform *is* the runtime.
- **Batteries included.** Components, design system, router, forms, SSR, add-ons — one vendor.
- **Deploy anywhere.** Static host, Python, or nginx; optional single-file bundle.

The one-sentence version: Nexa trades the npm ecosystem for a zero-dependency, no-build runtime — a deliberate reduction of attack surface and long-term maintenance cost, at the price of a smaller third-party component market (see the honest trade-offs in §07).

02 The security case

Your attack surface is what you ship. Nexa ships the framework and nothing else.

A typical SPA pulls hundreds of npm packages plus a build toolchain that **executes code at install time** — each a supply-chain entry point. Nexa has none of it: vendored ES modules, nothing to install.

DIMENSION	TYPICAL SPA STACK	NEXA
Runtime dependencies	tens-hundreds (transitive)	0
Build / dev dependencies	hundreds (bundler, transpiler, plugins)	0 — no build
Install-time code execution	yes (npm lifecycle scripts)	none — no install
Dynamic code (eval)	varies by dependency	none in the framework
What you audit	node_modules + lockfile churn	a few readable modules you vendor

Don't take our word for it — verify it

```
verify · reproducible from a clone

$ cat package.json | grep '"dependencies"'
"dependencies": {}    → 0 deps · nothing to install
$ grep -rE "from ['\"](https?:[a-z@])" dist/*.js
(no output)          → 0 external imports
$ grep -rE "\\beval\\(|new Function\\" dist/*.js
(no output)          → runs under strict CSP
```

Strict CSP, out of the box

No runtime code generation, so a hardened policy needs no unsafe-eval. Handlers use addEventListener, not inline attributes.

```
Content-Security-Policy:
  script-src 'self'; // no 'unsafe-eval'
```

Safe rendering by default

SSR output is HTML-escaped. A first-party guard neutralizes javascript:/data: URL attacks:

```
import { safeUrl } from
  '/dist/nexa.js';
h('a', { href: safeUrl(user.site) },
  'Visit')
```

Distribution & integrity. Delivered as vendored files or a CDN pinned to a release. For the strongest posture, pin to a commit and add a Subresource Integrity (integrity) hash — the browser then refuses any file that doesn't match.

03 The architecture case

No build — the app *is* the deployable

A Nexa app is an `index.html` and ES modules. Development is “save & refresh”; there is no compile step to wait on, debug, or secure. What you write is what runs and what ships — source maps are unnecessary because there is no transform.

```
<!doctype html>
<class="tok-f">link rel="stylesheet"
href="/dist/nexa-ui.css">
<class="tok-f">div id="app"></class="tok-f">div>
<class="tok-f">script type="module"
src="./app.js"></class="tok-f">script>
```

That is a complete, production-shippable Nexa page.

Familiar mental model

A React-trained engineer is productive immediately: components are functions, state and effects are hooks. The runtime is simpler (eager evaluation, subtree re-render) but the ergonomics are the ones your team already knows.

```
import { h, useState, useEffect } from
'/dist/nexa.js';

function Counter() {
  const [n, setN] = useState(0);
  useEffect(() => { document.title =
`${n}`; }, [n]);
  return h('button', { onClick: () =>
setN(v => v + 1) },
    `Clicked ${n}`);
}
```

Longevity & low lock-in

There is no bundler major-version to chase, no framework migration treadmill. Nexa targets the browser's own module system, so an app written today keeps running as long as ES modules do. If Nexa itself ever stalled, you already hold the full MIT source — it can't rot the way an unmaintained dependency tree does.

Batteries included, one vendor

Instead of assembling and securing a dozen libraries, one vendored tree covers the surface:

~60 components

Design system + theming

Router

Forms

SSR + hydration

i18n

a11y-audited

Presentations

Animation

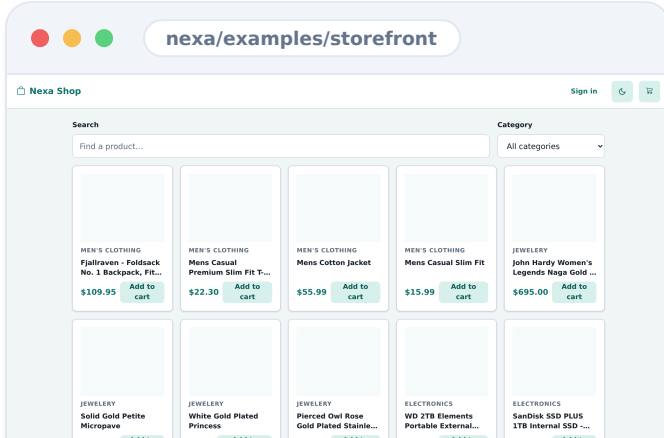
Diagrams

Code editor

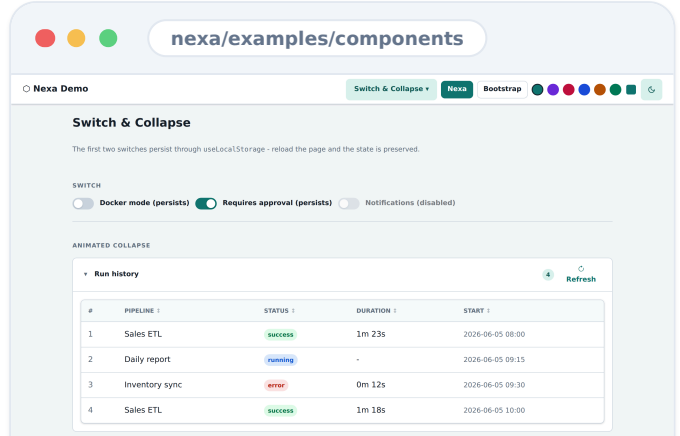
Fits a Python or ops-light shop. All of Nexa's own tooling — dev server, test runner, validators, optional bundler — is Python standard library. There is no Node to install on developer machines, in CI, or on servers. Deployment is static files behind any host, a Python process, or nginx.

04 Built with Nexa

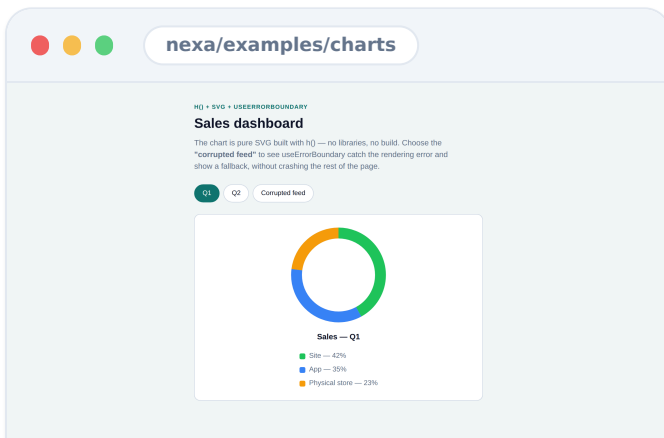
These are real example apps in the repository, captured in a headless browser. Every one is static files with no build step — the same components, design tokens and add-ons your team would use.



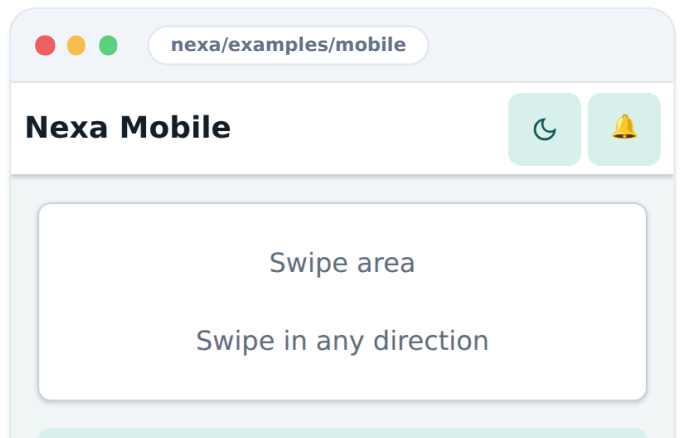
E-commerce storefront — product grid, live search, category filter, cart and dark-mode toggle from the component library.



Design system — switches, sortable tables, animated collapse, plus live theme (Nexa/Bootstrap) and palette switching.



Data dashboard — charts drawn as pure SVG with `h()`, no charting library, with an error boundary catching a bad feed.



Mobile shell — app bar, bottom nav, FAB, gesture hooks (swipe / long-press) and safe-area insets, mobile-first grid.

05 Developer experience

Familiar to any React-trained engineer, but simpler underneath. A few snippets show the whole surface — no JSX, no build, real ES modules.

A validated form

Built-in useForm + a field component:

```
const { field, handleSubmit, isSubmitting } = useForm({
  initialValues: { email: '' },
  validate: (v) => ({
    email: v.email.includes('@') ? '' :
    'Invalid',
  }),
  onSubmit: async (v) => { await
    api.login(v); },
});

h('form', { onSubmit: handleSubmit() },
  h(TextField, { ...field('email'),
    label: 'E-mail' }),
  h(Button, { type: 'submit', disabled:
    isSubmitting },
    'Sign in'));
```

Server-side rendering

Same components; escaped output; hydrate on the client:

```
import { renderToString, hydrate }
  from '/dist/nexa-server.js';

// server: safe, escaped HTML string
const html = renderToString(App);

// client: adopt the server DOM in place
hydrate(App,
  document.getElementById('app'));
```

Themeable by tokens

```
// dark mode + palette, no provider
needed
const { toggleTheme } = useTheme();
const { setPalette } = usePalette();
```

Everything on the cover is this stack. The zoom presentation, the synthesizer panel and the component gallery are all plain modules like these — the visual add-ons (presentations, animation, diagrams, a code editor) use the same `h()` and hooks, just imported when you need them.

06 Engineering maturity

“No dependencies” must not mean “untested.” Nexa is validated the way a platform should be — and, keeping the philosophy intact, entirely with Python + a real browser, no Node.

~300 tests

Engine, hooks, components, SSR, a11y, add-ons and security — asserted against a real DOM.

3 engines

The suite runs on Chromium, Firefox and WebKit in CI on every change.

Static gates

A validator checks imports resolve, assets exist, no monoliths, versions sync, minified files match.

Continuous integration

- Static validation + build-integrity checks (minified/derived files can't drift).
- Full browser suite across three engines, fail-fast off (all engines reported).
- All jobs are python3 — the CI needs no npm registry access.

Documented QA surface

- A QA runbook and an ID'd scenario catalog (~180 cases) cover every hook, component and add-on.
- An AI-generation spec keeps generated code correct and idiomatic.
- Accessibility is tested (focus traps, roving tabindex, ARIA), not assumed.

Auditability as a feature. The core runtime is a few thousand lines of readable, unminified ES modules. A security reviewer can read the entire framework end-to-end in an afternoon — something that is simply not possible with a conventional dependency tree.

07 Honest trade-offs

A brief that only lists strengths doesn't survive an architecture review. Here is where Nexa costs you something, and how those costs are bounded.

✓ Where Nexa fits best

- Internal tools, dashboards, admin panels, line-of-business apps.
- Environments where supply-chain risk, air-gapping or long support windows matter.
- Teams without (or wanting to shed) a Node build pipeline; Python/ops-light shops.
- Projects that value auditability and longevity over ecosystem breadth.

△ Where to think twice

- You need a specific large third-party React/Vue component (maps, rich grids) — Nexa's market is smaller.
- Org standards mandate a particular framework or hiring pool.
- You rely on build-time features (heavy tree-shaking of a huge app, TS-first workflows) — Nexa favors runtime simplicity.

CONCERN A REVIEWER WILL RAISE

HONEST RESPONSE

"Smaller / newer project, single maintainer."

True. Mitigated structurally: it's MIT and **vendored**, so you own the source; with zero dependencies there is no tree to rot, and the whole runtime is auditable and forkable. Your continuity doesn't depend on the upstream staying active.

"No npm ecosystem of components."

Correct — that's the deliberate trade. The built-in ~60 components + add-ons cover most app UI; anything else is plain DOM/Web Components, which interop directly.

"No TypeScript build."

Ships hand-written `.d.ts` type declarations for editor/tooling support; you get types without a compile step, though not a TS-first authoring flow.

"Is no-build actually performant?"

For most apps, yes — HTTP/2 + a pinned CDN handle module loading; an **optional** Python/esbuild bundler produces a single-file deploy when you want it, still with no Node.

08 Deploy, integrate & next step

Deployment options

- **Static host / CDN** — copy the folder; pin the CDN to a tag + SRI.
- **Python or nginx** — for SSR, SPA-fallback routing, or gzip/brotli + immutable caching.
- **Single-file bundle** — optional, for one-request deploys; still zero Node.
- **Air-gapped** — everything is vendored; nothing phones home.

Integrates with what you have

- Backend-agnostic — talks to any HTTP/JSON API (your existing services).
- SSR entry runs on any server runtime for first-paint + SEO.
- Drops into an existing page (mount into a `<div>`); adoptable incrementally.
- Composes with native Web Components and plain DOM.

RECOMMENDED NEXT STEP

Pilot it on one internal tool. Pick a low-risk dashboard or admin screen, build it with Nexa, and put it through your standard security review and the reproducible checks in §02. You'll evaluate the real thing — zero-install, CSP-clean, auditable — against your own bar, with no toolchain to stand up first.

VERIFY

Reproduce every security claim from a clone — see §02.

READ

The full runtime is unminified, MIT, and auditable end-to-end.

OWN

Vendor it. No registry, no lockfile, no install step in your pipeline.

Nexa

Prepared for internal security & architecture evaluation · Framework is MIT-licensed · Claims in this brief are reproducible from the source repository.